

Outline · read a page one part at a time

1 How a line says which part it belongs to

Two ways, and the first one is free: most lines already say it; the mark is for the rest.

```
(1) already said      ### A3 group . id A3.1      this is part 3
    (nothing to do)   ### P  group . id P1        this is the whole page
```

```
(2) the SN mark . WHERE it sits is what makes it a mark
    Aims      - [x]  S1 the text ...      at the FRONT
    States    S1 shipped 260815 ...      at the FRONT
    Files     `build.py` S1 S2          at the END, may be several
    Log       never marked              it is a diary, not a part
```

```
these look the same but are not marks, all seen on real pages
`QB6` S7 . [QB7 S3](...)  a part of ANOTHER page
S5.1 . S4.3              a smaller piece inside part 5
SRequired Inputs         a name, not a number
"Every number S4 prints" a paper's own section, in a sentence
```

Most lines already say which part they belong to; for the rest you write a small mark like §2. A page written the normal way needs no mark at all. Its goals are already grouped ### A3 and numbered A3.1, and both mean part 3. P1 already means the whole page. This page is its own proof: every one of its goals lands in the right part, with no mark written anywhere. The mark is for what that leaves out: pages written before the numbering, and stray lines that sit outside any group. The mark is § and a number, and the number is a part of THIS page's own Content. WHERE the mark sits is what makes it a mark. The § sign was used on these boards long before this tab, and a paper page writes §4 in normal sentences to mean the paper's own section 4. Nothing about the characters can tell those apart. So a mark is only read in a place where a sentence would not put one. On a goal or a progress line it sits at the front, before the words. On a file line it sits at the end, and there may be several, because one file often serves several parts. A §4 in the middle of a sentence is just writing, and is left alone. A line with no mark belongs to the whole page. So an old page with no marks still reads fine; it just shows as one card, which is honest but tells you little. Diary lines in ## Log never take a mark, because cutting a diary into parts loses the story. The mark points by number, so whoever rennumbers the parts has to fix the marks in the same edit. A mark pointing past the last part is caught and named. A mark pointing at the wrong part is not, because it still points at a real one.

2 The tab, read fresh every time you open it

One read does both jobs: the same pass checks the page and draws it.

```
-- what you see first -----
QPf11-pagex
How does a page borrow a file from another page ...?      the page's
11 of 12 done . [ ] 1 to do . 1 waiting      own question
```

```
-----
[x] every line is placed . or 3 lines not placed in any part
[ By part] [ What is left]
```

```
-- one card -----
1 . The contract 2/2
what a person writes and keeps one line about the part
[ ] STILL TO DO goals not done, always in sight
[x] 2 DONE finished goals, folded away
MORE notes and files, folded away
```

```
-- one goal -----
the goal plain words . the id once, small, on the right
NOW ... its progress, under it
```

The tab reads the page every time you open it, so it can never be out of date. The tab is drawn by GET /_board/outline and saved nowhere. This copies QP1: anything written to disk starts going out of date the moment it lands. Because nothing is saved, this plugin has no stale flag, no rebuild button, and no folder of its own. It is not on the plugin list either, because it owns no folder to be on the list for. Checking is not a separate step. A mark pointing past the last part shows as a red on the card it tried to reach. Every unmarked goal or progress line lands on the card, so that card is also the list of lines still worth marking. The two views are the same lines sorted twice. Clicking a chip re-sorts them in the browser, with nothing fetched again. What is left puts unfinished goals first, because that is what you opened it to ask. THE ANSWER COMES FIRST, and that is a requirement, not a taste. This tab is read by someone with ADHD, in a language that is not their first. The first version failed both. It opened straight into part 1, with no word about what the page was even for. Every visible line was a long sentence. Finished work was printed in full, above the one thing still to do. So the top of the tab now carries the page's own opening question. Under it sits one line of numbers: how many goals, how many done, how many left, how many questions wait on a person. Each card carries its own count, and one line saying what its part is about. Unfinished goals stay in plain sight; finished ones fold away behind a count you can click. Nothing is hidden and nothing is summarized by a machine. What changed is the ORDER, so one screen tells you where the page stands. The words changed too. **loose lines** became **lines not placed in any part**, and **By division** became **By part**. A reader opening the tab for the first time has no reason to know this plugin's own shorthand. A goal and its progress are two different sentences, and the card has to keep them looking different. A goal says what should become true. Progress says what is true now. Printed as two alike lines, both starting with A1.1 and both ticked, they read as one sentence said twice. So the id is printed once, small and grey on the right, and the progress sits under the goal behind a **now** label. Each card also carries one line saying what its part is about, and that line is **READ**, never written. The page template already asks each part for one: the line under its picture. Pages write it in their own hand, **Establishes the reading protocol**, so what the tab looks for is the **SHAPE**, an emoji then a sentence. A part with no such line falls back to its picture caption, then to its first plain sentence. A part with none of the three shows nothing, because a wrong summary costs the reader more than a missing one.

3 Getting older pages ready

AI shows up once, then leaves: it suggests the marks, a person checks them, and the file keeps them.

```
AI reads the page --> suggests a SN per line --> a person checks
                                     |
after that, the tab just reads what is written <-----+ saved in the page
```

Old pages get their marks once, with help, and never need it again. Pages written before the mark existed have none, so each shows as a single card. Fixing that is a one-time pass per page. An AI suggests a mark for each goal and progress line, the suggestion is written into the page as ordinary text, and a person reads the change before it lands. After that pass the answer lives in the page, and no AI runs when you open the tab. New pages take their marks as they are written, because whoever writes a goal knows which part it serves.

4 The plan is a FILE now, and OUTLINE is a phase

Two rulings on 260817: the plan stopped being something read out of the page, and started being something written before it.

```
until 260816 the tab DERIVED a plan from the page's own headings
              a plan could not exist before the page did
[x] from 260817 <page>/outline/<stem>-outline-v<N>.md AUTHORED . versioned
              exits only when a person ticks `approved:`
```

A plan you can reject in ten seconds now sits in front of every expensive phase. The phase contract is `page-workflows/haipipe-page-outline 0.1.0`, and it took the outline authority OUT of `haipipe-page-draft`, whose § had owned it. This page had ruled the opposite on 260816, in these words: "there is no outline phase and no outline file". What overturned it was a real failure the same week: one phase owned both agreeing the shape and writing the page, so one done-report covered both, and a plan ended up pasted into a page's own **## Content** where it went stale at the next edit. ****A bullet is a POINT, not a sentence**** (JL 260817). Its address is `C<n>.P<n>.B<n>`, and one B becomes one or more S when it is drafted, so the plan and the page count different units and say so. `C<n>.P<n>` is shared with the sentence address, so the link survives; C stays because Aims use A and page-wide uses P in that same slot. The C prints once on the section heading and the rows carry only `P<n>.B<n>`. ****A bullet with no mark is the normal case.**** Requiring a mark on every line made the plan unreadable and buried the few lines that owed something, so a mark is now the exception, written at the END of a bullet where the card will hang on the real sentence. Six marks: aim · [x] have it · citation · value · display · proof.

5 What the tab shows now, and the two halves it had never labelled

The join runs both ways: what the plan promised, against what is on disk.

```
THE PLAN      the file . one row per bullet . each ref opens as a card
THE PAGE NOW  folded . the page's own divisions, as written today
```

Both halves were on screen unlabelled, and a reader could not tell them apart. The tab reads three things now: the plan file, the page .md, and the page's sibling plugin folders. It still writes nothing, caches nothing and calls no model; storage-less was the old invariant and only "markdown-only" was given up. Each , and renders as the board's own evidence card: a chip in the line and a popover holding the THING itself, the reference as printed, the card's own question, the unit's own claim. Three counts are computed separately and never collapsed: **owed**, **landed**, **accepted**. The reverse direction is drawn too: a card on disk that no bullet cites shows as , which caught its own first defect within a minute of shipping. THE PAGE NOW folds shut, because on a page whose aims carry no ids it was nine empty cards in the way.